

# TP — Aprendizizado Contínuo com Controle por Reforço

Projeto Independente de Pesquisa

Entrega: Código, Relatório curto, Slides

3 de novembro de 2025

## 1 Visão geral

O **aprendizado contínuo** (ou *continual learning*) busca construir modelos que aprendem sequencialmente, incorporando novas tarefas sem esquecer as anteriores. O principal desafio é o *catastrophic forgetting*, isto é, a perda de desempenho em tarefas já aprendidas ao adquirir novos conhecimentos.

Projetos clássicos atacam esse problema por meio de regularizações fixas (como EWC e SI) ou jogos min-max adaptativos. Neste projeto, propomos uma abordagem alternativa: modelar o equilíbrio **estabilidade–plasticidade** como um **problema de controle sequencial** resolvido via **Aprendizado por Reforço (RL)**.

A ideia é que um **agente** observe métricas do treinamento (como acurácias, perdas e deriva dos parâmetros) e **decida dinamicamente** hiperparâmetros de regularização, taxa de *replay* ou congelamento de camadas, buscando maximizar o desempenho em novas tarefas sem violar limites de esquecimento.

## 2 Objetivos

### Objetivo geral

Investigar o uso de *Reinforcement Learning* como controlador adaptativo da estabilidade–plasticidade em aprendizado contínuo, visando reduzir o esquecimento catastrófico e otimizar a retenção do conhecimento em cenários sequenciais.

### Objetivos específicos

1. Formular o aprendizado contínuo como um problema de controle com restrição de esquecimento.
2. Implementar um agente RL (bandit contextual e ator–crítico) para ajustar  $\lambda_t$  (penalização de deriva) durante o treinamento.
3. Comparar desempenho, estabilidade e eficiência com baselines fixos e métodos regulares (EWC, SI).
4. Explorar variantes de controle: seleção adaptativa de taxa de *replay* e congelamento seletivo de camadas.

## 3 Formulação como CMDP

Modelamos o processo de aprendizado como um *Constrained Markov Decision Process* (CMDP):

**Estados.** Vetores de métricas observadas:

$$s_t = (\text{acc}_A, \text{acc}_B, \mathcal{L}_A, \mathcal{L}_B, \|\theta - \theta_A\|_2, \text{época}, \text{grad-norms}).$$

**Ações.**

$$a_t = (\lambda_t, \rho_t),$$

onde  $\lambda_t \geq 0$  regula a penalização de deriva e  $\rho_t \in [0, 1]$  é a fração de *replay* opcional.

**Recompensa e custo.**

$$r_t = \Delta \text{acc}_B, \quad c_t = \max(0, \text{acc}_A^{(t-1)} - \text{acc}_A^{(t)}).$$

**Objetivo.** Maximizar o retorno esperado sob uma restrição de custo:

$$\max_{\pi} \mathbb{E}_{\pi} \left[ \sum_t r_t \right] \quad \text{sujeito a} \quad \mathbb{E}_{\pi} \left[ \sum_t c_t \right] \leq \tau.$$

**Lagrangiano.**

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{\pi} \left[ \sum_t (r_t - \lambda c_t) \right], \quad \lambda \leftarrow [\lambda + \eta_{\lambda}(\bar{c} - \tau)]_+.$$

## 4 Metodologia

### Parte 1 — Bandit contextual (simples)

Cada época é um episódio. O agente observa o estado ( $\text{acc}_A, \text{acc}_B, \|\theta - \theta_A\|_2$ ) e escolhe  $\lambda_t$  de uma grade discreta. A recompensa é o ganho de  $\text{acc}_B$  penalizado pelo esquecimento em A. O agente é treinado via Thompson Sampling ou UCB.

### Parte 2 — Ator-crítico com restrição

Treinamento contínuo com ator-crítico (PPO/A2C). A política  $\pi_\phi(a_t|s_t)$  aprende a ajustar  $\lambda_t$  e  $\rho_t$ , enquanto o multiplicador  $\lambda_{\text{dual}}$  é atualizado online para respeitar a restrição de esquecimento  $\tau$ .

### Parte 3 — Comparações e métricas

- Baselines:  $\lambda$  fixo, EWC, SI.
- Métricas:  $\text{acc}_A$ ,  $\text{acc}_B$ , esquecimento total, deriva média, retorno acumulado.
- Gráficos: trajetória de  $\lambda_t$ , custo acumulado vs. limite  $\tau$ , curva Pareto (esquecimento  $\times$  desempenho em B).

### Como rodar (protótipo)

```
python rl_controller.py --epochs 10 --agent bandit --lambda_grid 1e-4 1e-3 1e-2
python rl_controller.py --agent actor_critic --theta_lr 1e-3 --policy_lr 3e-4
```

### Apêndice A — Esqueleto simplificado do agente RL

```
# rl_controller.py
# Minimal example: bandit controlling lambda_t

for epoch in range(E):
    state = collect_metrics(model)
    lambda_t = agent.act(state)
    loss_B = CE(model(x_B), y_B)
    theta = flatten_params(model)
    F_drift = 0.5 * torch.sum((theta - theta_A)**2)
    obj = loss_B + lambda_t * F_drift
    backprop(obj)

    accA_new, accB_new = evals(...)
    reward = accB_new - accB_prev
    cost = max(0, accA_prev - accA_new)
    agent.update(state, lambda_t, reward - dual_lambda * cost)
    dual_lambda = max(0, dual_lambda + eta * (cost - tau))
```

## A Resultados esperados

Espera-se que o controlador RL:

- Ajuste  $\lambda_t$  dinamicamente, evitando tanto esquecimento quanto rigidez excessiva;
- Supere penalizações fixas em termos de estabilidade e desempenho;
- Produza curvas de custo e acurácia mais estáveis, aproximando-se do limite Pareto ótimo entre retenção e aprendizado.

## B Conclusão

Este projeto propõe uma integração direta entre aprendizado contínuo e aprendizado por reforço. O agente atua como um controlador adaptativo, aprendendo políticas de regularização que balanceiam estabilidade e plasticidade de forma autônoma. A abordagem abre caminho para sistemas de aprendizado contínuo verdadeiramente autoajustáveis.